Functional Dependencies in DBMS

Functional Dependencies in DBMS

A **functional dependency (FD)** is a relationship between two attributes in a relation (table). It describes how one attribute uniquely determines another. If attribute `A` determines attribute `B`, it is represented as:

A→BA \rightarrow BA→B

This means that for any two rows in the table, if the value of `A` is the same, the value of `B` must also be the same.

Types of Functional Dependencies

1. **Trivial Dependency**:
   If A→BA \rightarrow BA→B, and B⊆AB \subseteq AB⊆A, it is trivial.
   Example: {A,B}→A\{A, B\} \rightarrow A{A,B}→A
2. **Non-Trivial Dependency**:
   If A→BA \rightarrow BA→B, and B⊆/AB \not\subseteq AB⊆/A, it is non-trivial.
   Example: A→BA \rightarrow BA→B
3. **Transitive Dependency**:
   If A→BA \rightarrow BA→B and B→CB \rightarrow CB→C, then A→CA \rightarrow CA→C.
   Example: In a table with `StudentID → CourseID` and `CourseID → Instructor`, we infer `StudentID → Instructor`.

Normalization in DBMS

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves dividing a database into tables and defining relationships to ensure minimal duplication.

Normalization Forms

1. **First Normal Form (1NF)**:
   - Ensures that all attributes have atomic (indivisible) values.

- No repeating groups or arrays.
  Example:
  Before 1NF:
StudentID	Subjects
1	Math, Science

2. After 1NF:

StudentID	Subject
1	Math
1	Science

3. **Second Normal Form (2NF)**:
   - Achieves 1NF and ensures no partial dependency (non-prime attributes depend only on a part of the primary key).
     Example:
     A table with composite key {StudentID,CourseID}\{StudentID, CourseID\}{StudentID,CourseID}:
StudentID	CourseID	Instructor	StudentName
4. `StudentName` depends only on `StudentID`, not the full key.
   After 2NF:
   - Separate tables for `Student` and `CourseInstructor`.
5. **Third Normal Form (3NF)**:
   - Achieves 2NF and ensures no transitive dependency.
     Example:
     A table with StudentID→CourseIDStudentID \rightarrow CourseIDStudentID→CourseID and CourseID→InstructorCourseID \rightarrow InstructorCourseID→Instructor:
     | StudentID | CourseID | Instructor |
6. After 3NF:
   - Separate tables for `StudentCourse` and `CourseInstructor`.
7. **Boyce-Codd Normal Form (BCNF)**:
   - A stricter version of 3NF, ensuring every determinant is a candidate key.

Example: Functional Dependency and Normalization

Unnormalized Table:

StudentID	StudentName	CourseID	Instructor	Department

1	Alice	CS101	Dr. Smith	CS
2	Bob	CS101	Dr. Smith	CS
3	Charlie	MA101	Dr. Lee	Math

Functional Dependencies:

1. StudentID→StudentNameStudentID \rightarrow StudentNameStudentID→StudentName
2. CourseID→Instructor,DepartmentCourseID \rightarrow Instructor, DepartmentCourseID→Instructor,Department

Normalization Process:

1. **1NF**:
   No changes needed (atomic values).
2. **2NF**:
   Remove partial dependencies:
   - Table 1: StudentID→StudentNameStudentID \rightarrow StudentNameStudentID→StudentName
   - Table 2: CourseID→Instructor,DepartmentCourseID \rightarrow Instructor, DepartmentCourseID→Instructor,Department

3.

StudentID	**StudentName**
1	Alice
2	Bob
3	Charlie

4.

CourseID	**Instructor**	**Department**
CS101	Dr. Smith	CS
MA101	Dr. Lee	Math

5. **3NF**:
   No transitive dependencies, so no further changes.

4th Normal Form (4NF)

- A table is in **4NF** if it is in **Boyce-Codd Normal Form (BCNF)** and has no **multi-valued dependencies**.
- **Multi-valued dependency** occurs when one attribute in a table determines multiple values of another attribute, independent of other attributes.

Example:
A table showing students enrolled in multiple courses and participating in multiple activities:

StudentID	Course	Activity
1	Math	Basketball
1	Math	Chess
1	Science	Basketball
1	Science	Chess

Issues:

- Redundancy occurs because courses and activities are independent of each other.

Solution (4NF):
Decompose the table into two:

StudentID	Course
1	Math
1	Science

StudentID	Activity
1	Basketball
1	Chess

Boyce-Codd Normal Form (BCNF)

- A stricter version of 3NF.
- A table is in **BCNF** if it is in **3NF** and **every determinant is a candidate key**.

Example:
Consider a table where a student is assigned to a professor for a specific subject:

StudentID	Subject	Professor
1	Math	Dr. Smith
2	Science	Dr. Lee
1	Science	Dr. Smith

Functional Dependencies:

1. StudentID,Subject→ProfessorStudentID, Subject \rightarrow ProfessorStudentID,Subject→Professor
2. Professor→SubjectProfessor \rightarrow SubjectProfessor→Subject

Issue:

- Professor→SubjectProfessor \rightarrow SubjectProfessor→Subject violates BCNF because `Professor` is not a candidate key.

Solution (BCNF):
Decompose the table:

Professor	Subject
Dr. Smith	Math
Dr. Lee	Science

StudentID	Subject
1	Math
2	Science

5th Normal Form (5NF) / Project-Join Normal Form (PJNF)

- A table is in **5NF** if it is in **4NF** and **all join dependencies are lossless**.
- 5NF deals with decomposing tables into smaller tables to eliminate redundancy while ensuring the original table can be reconstructed through joins.

Example:
Consider a table showing students, courses, and instructors:

StudentID	Course	Instructor
1	Math	Dr. Smith
1	Science	Dr. Lee
2	Math	Dr. Smith

Issues:

- Redundancy arises because there is no direct dependency between `StudentID`, `Course`, and `Instructor`.

Solution (5NF):
Decompose the table into three:

StudentID	Course
1	Math
1	Science
2	Math

Course	Instructor
Math	Dr. Smith
Science	Dr. Lee

StudentID	Instructor
1	Dr. Smith
1	Dr. Lee
2	Dr. Smith

Summary of Normal Forms

Normal Form	Key Characteristics
1NF	Eliminates multi-valued attributes; ensures atomic values.
2NF	Removes partial dependencies; all non-prime attributes depend on the whole primary key.
3NF	Removes transitive dependencies; non-prime attributes depend only on candidate keys.
BCNF	Ensures every determinant is a candidate key.
4NF	Eliminates multi-valued dependencies.
5NF	Ensures all join dependencies are lossless; eliminates redundancy caused by complex relationships.

Each higher normal form builds on the principles of the previous one, ensuring better data integrity and reduced redundancy.